Использование ссылок и ref-структур для низкоуровневой оптимизации C# кода

Юрий Малич

yurymalich@yandex.ru

План

- История развития поддержки модификатора ref в .NET
- Операции с управляемыми указателями
- ref-структуры, Span<T> и работа с ним
- Использование ref-структур и управляемых указателей под капотом .NET runtime
- Структура Memory<T>

О себе

- Закончил ПГУПС (Автоматика и телемеханика на ж/д транспорте)
- Программирую примерно с 1996 года, с MS DOS на Intel i386 (Pascal, C++, ASM)
- На C# .NET с первых версий. Microsoft Certified Professional 2016
- Писал статьи по архитектуре микропроцессоров для сайтов iXBT.com, fcenter.ru, 3dnews.ru
- Принимал участие в разработке системных утилит для Windows: в Nero AG (Nero Burning Rom) и в TuneUp (TuneUp Utilities)
- Разрабатываю Desktop и Backend приложения на C# .NET

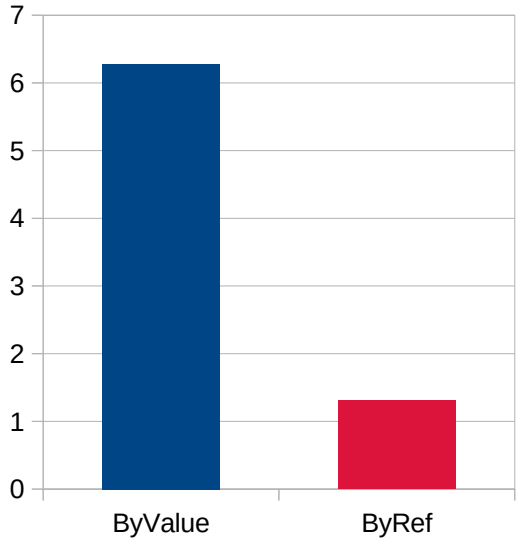
ref в C# 1.0

```
struct StructData
{
    public long A1;
    public long A2;
    public long A3;
    public long A4;
}

public virtual StructData Transform(StructData rect)
{
    rect.A1++;
    return rect;
}

StructData item1 = new StructData();

item1 = Transform(item1);
```

ref в C# 1.0

```
public virtual void Transform(ref StructData rect)
{
    rect.A1++;
}

StructData item1 = new StructData();

Transform(ref item2);
```

Method	Mean	Ratio	Code Size
ByVal	6.283 ns	1.00	58 B
ByRef	1.313 ns	0.21	26 B



ref в C# 7 (2017) - managed pointer

```
int x = 42;
ref int refX = ref x;
refX = 0; // x = 0;

int[] array = [11, 12, 13, 14];

ref int refArray1 = ref array[1]; // 12

ref int refArray2 = ref array[2]; // 13

refArray1 = 42; // array[1] = 42;

refArray1 = ref refArray2 ;
// ref int refArray1 = ref Array[2];
```

ref в C# 7 (2017) - managed pointer

```
ref byte GetArrayRef(int i)
{
    byte[] bytes = new byte[] { 0, 1, 2, 3, 4, 5, 6, 7, 8, 9 };
    return ref bytes[i];
}

ref byte x = ref GetArrayRef(2); // bytes[2] == 2

x = 10;
// { 0, 1, 10, 3, 4, 5, 6, 7, 8, 9 }; // bytes[2] == 10

GC.Collect(2, GCCollectionMode.Forced);

Console.WriteLine($"x = {x}"); // ????
// «x = 10»

// { 0, 1, 10, 3, 4, 5, 6, 7, 8, 9 };
```

Managed interior pointers

```
ref byte x = ref GetArrayRef(2);
mov qword ptr [rbp+88h],rax

x = 10;
mov         rax,qword ptr [rbp+88h] // RBP+88h = 0x00000040D9B7EA28
mov         byte ptr [rax],0Ah  // rax = 0x000001AAB9805182
```

Memory 2

Address: 0x00000040D9B7EA28

```
0x00000040D9B7EA28  000001aab9805182 00000040d9b7ecd8 00000040d9b7ee78 00000040d9b7ea50 00007ff86e7d8c1e
0x00000040D9B7EA50  00000000ffffffff 000001aab9804dc8 000001aab9804790 00000000ffffffff 00000040d9b7ec50
```

Memory 3

Address: 0x000001AAB9805180

```
0x000001AAB9805180  00 01 0a 03 04 05 06 07 08 09 00 00
0x000001AAB98051A4  00 00 00 00 00 00 00 00 00 00 00 00
```

```
= byte[] { 0, 1, 10, 3, 4, 5, 6, 7, 8, 9 }
```

Managed interior pointers

```
GC.Collect(2, GCCollectionMode.Forced);
```

```
x => [rbp+88h] // = 0x0000001AABBC598C2 (было 0x0000001AAB9805182)
```

Memory 2

Address: 0x00000040D9B7EA28

```
0x00000040D9B7EA28  000001aabbc598c2 00000040d9b7ecd8 00000040d9b7ee78 00000040d9b7ea50 00007ff86e7d8c1e
0x00000040D9B7EA50  00000000ffffffff 000001aab9804dc8 000001aab9804790 00000000ffffffff 00000040d9b7ec50
```

ë

Memory 3

Address: 0x000001AABBC598C0

```
0x000001AABBC598C0  00 01 0a 03 04 05 06 07 08 09 00
0x000001AABBC598E4  43 00 3a 00 5c 00 50 00 72 00 6f
```

```
= byte[] { 0, 1, 10, 3, 4, 5, 6, 7, 8, 9 }
```

Managed interior pointers

```
class ColorArgb
{
    public byte B;
    public byte G;
    public byte R;
    public byte A;
}

ref byte GetClassFieldRef()
{
    ColorArgb x = new ColorArgb();
    return ref x.R;
}

ref byte r = ref GetClassFieldRef();
r = 0xFF;
GC.Collect(2, GCCollectionMode.Forced);
// r == 0xFF
```

Scoped ref

```
struct ColorArgb
{
    public byte B;
    public byte G;
    public byte R;
    public byte A;
}

ref byte GetStructRef()
{
    ColorArgb testStruct = new ColorArgb();
    return ref testStruct.B;
    // Cannot return a member of local 'testStruct' by reference because it is not a ref local

    [ScopedRef]
    ref byte b = ref testStruct.B;
    return ref b;
    // Cannot return 'b' by reference because it was initialized to a value that cannot be
     returned by reference
}
```

public static class Unsafe

Disclaimer

⚠ **Warning**

This type is intended for advanced pointer manipulation scenarios. Callers are assumed to be familiar with ECMA-335, Sec. II.14.4 and III.1.1.5, and with the ECMA-335 CLI Specification Addendum.

Most APIs on this type do not perform any argument checking or validation. Incorrect use of these APIs could corrupt process memory or destabilize the .NET runtime.

*This type is typically used by **low-level library** authors who want to write **high-performance code** and are willing **to suppress** .NET's typical **type safety checks to meet their performance goals**.*

class Unsafe

```
ref T Unsafe.Add<T>(ref T source, int elementOffset)
```

```
int[] buffer = new int[10] { 0, 1, 2, 3, 4, 5, 6, 7, 8, 9 };

ref int refBuffer1 = buffer[1];

ref int refBuffer3 = ref Unsafe.Add(ref refBuffer1, 2); // 1 + 2 = 3

[
    0,
    1, <= refBuffer1
    2,
    3, <= refBuffer3
    …,
];

RefBuffer3 = 42; //  buffer[3] = 42;
```

class Unsafe

```
ref T Unsafe.Add<T>(ref T source, int elementOffset)
```

```
public int[] QuickSortIndex(int[] array, int leftIndex, int rightIndex)
{
    int i = leftIndex;
    int j = rightIndex;
    int pivot = array[leftIndex];
    while (i <= j)
    {
        while (array[i] < pivot) i++;
        while (array[j] > pivot) j--;
        if (i <= j)
        {
            int temp = array[i];
            array[i] = array[j];
            array[j] = temp;
            i++; j--;
        }
    }

    if (leftIndex < j) QuickSort(array, leftIndex, j);
    if (i < rightIndex) QuickSort(array, i, rightIndex);

    return array;
}
```

class Unsafe

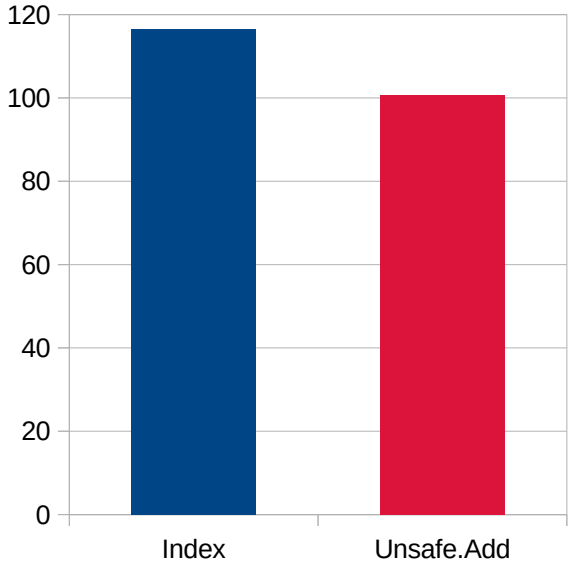
ref T Unsafe.Add<T>(ref T source, int elementOffset)

```
public int[] QuickSortUnsafeAdd(int[] array, int leftIndex, int rightIndex)
{
    int i = leftIndex;
    int j = rightIndex;
    int pivot = array[leftIndex];
    ref int arrayRef = ref array[0];
    while (i <= j)
    {
        while (Unsafe.Add(ref arrayRef, i) < pivot) i++; // while (array[i] < pivot) i++;
        while (Unsafe.Add(ref arrayRef, j) > pivot) j--; // while (array[j] > pivot) j--;
        if (i <= j)
        {
            int temp = Unsafe.Add(ref arrayRef, i); // int temp = array[i];
            Unsafe.Add(ref arrayRef, i) = Unsafe.Add(ref arrayRef, j); // array[i] = array[j];
            Unsafe.Add(ref arrayRef, j) = temp; // array[j] = temp;
            i++; j--;
        }
    }

    if (leftIndex < j) QuickSort2(array, leftIndex, j);
    if (i < rightIndex) QuickSort2(array, i, rightIndex);
    return array;
}
```

class Unsafe

QSort(): [Index] vs Unsafe.Add(ref, Index), 1 000 000 random ints

Method	Mean	Error	StdDev	Ratio	RatioSD	Code Size
QSortIndex	116.6 ms	2.31 ms	2.17 ms	1.00	0.03	443 B
QSortUnsafeAdd	100.7 ms	0.82 ms	0.77 ms	0.86	0.02	437 B



Unsafe.Add<T>()

`while (array[i] < pivot) i++;`	`while (Unsafe.Add(ref arrayRef, i) < pivot) i++;`
`M02_L02:`	`M02_L02:`
`inc edi`	`inc edi`
`cmp edi,ecx`	`----------`
`jae M02_L09`	`----------`
`mov r10d,edi`	`movsxd r10,edi`
`cmp [rbx+r10*4+10],eax`	`cmp [rax+r10*4],ecx`
`jl M02_L02`	`jl M02_L02`

`while (array[j] > pivot) j--;`	`while (Unsafe.Add(ref arrayRef, j) > pivot) j--;`
`M02_L04:`	`M02_L04:`
`dec r8d`	`dec r8d`
`cmp r8d,ecx`	`----------`
`jae M02_L09`	`----------`
`mov r10d,r8d`	`movsxd r10,r8d`
`cmp [rbx+r10*4+10],eax`	`cmp [rax+r10*4],ecx`
`jg M02_L04`	`jg M02_L04`

class Unsafe

```
ref T Unsafe.Add<T>(ref T source, int elementOffset)
```

```
private static void InsertionSort(Span<T> keys)
{
    for (int i = 0; i < keys.Length - 1; i++)
    {
        T t = Unsafe.Add(ref MemoryMarshal.GetReference(keys), i + 1);

        int j = i;
        while (j >= 0 && (t == null
          || LessThan(ref t, ref Unsafe.Add(ref MemoryMarshal.GetReference(keys), j))))
        {
            Unsafe.Add(ref MemoryMarshal.GetReference(keys), j + 1)
                    = Unsafe.Add(ref MemoryMarshal.GetReference(keys), j);
            j--;
        }

        Unsafe.Add(ref MemoryMarshal.GetReference(keys), j + 1) = t!;
    }
}
```

class Unsafe

ref T Unsafe.Add<T>(ref T source, int elementOffset)

```
internal static int CompareStringIgnoreCase(ref char strA, int lengthA, ref char strB, int lengthB)
{
    int length = Math.Min(lengthA, lengthB);
    int range = length;

    ref char charA = ref strA;
    ref char charB = ref strB;

    const char maxChar = (char)0x7F;

    while (length != 0 && charA <= maxChar && charB <= maxChar)
    {
        // Ordinal equals or lowercase equals if the result ends up in the a-z range
        if (charA == charB ||
            ((charA | 0x20) == (charB | 0x20) && char.IsAsciiLetter(charA)))
        {
            length--;
            charA = ref Unsafe.Add(ref charA, 1);
            charB = ref Unsafe.Add(ref charB, 1);
        }
        else
        {
```

class Unsafe

```
ref TTo Unsafe.As<TFrom, TTo>(ref TFrom source)
```

```
byte[] buffer = [1, 2, 3, 4, 5, 6, 7, 8];

ref byte refB = ref buffer[0]; // byte* x

ref int refInt = ref Unsafe.As<byte, int>(ref refB); // int* refInt

int val = refInt; // val = 0x04030201

refInt = 0x0F0E0C0B;
```

[0]	0x0b	byte
[1]	0x0c	byte
[2]	0x0e	byte
[3]	0x0f	byte
[4]	0x05	byte
[5]	0x06	byte
[6]	0x07	byte
[7]	0x08	byte

class Unsafe

```
ref TTo Unsafe.As<TFrom, TTo>(ref TFrom source)
```

.NET 9.0

```
public static byte[] BitConverter.GetBytes(long value)
{
    byte[] bytes = new byte[sizeof(long)];
    Unsafe.As<byte, long>(ref bytes[0]) = value;
    return bytes;
}
```

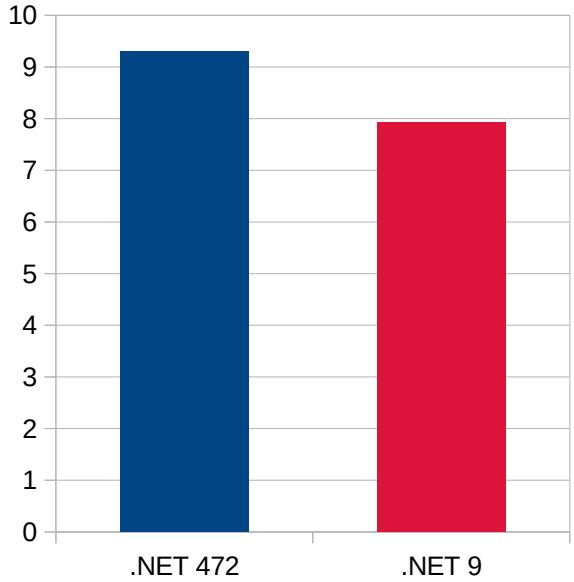
.NET Framework 4.7.2

```
public unsafe static byte[] BitConverter.GetBytes(long value)
{
    byte[] array = new byte[8];
    fixed (byte* ptr = array)
    {
        *(long*)ptr = value;
    }
    return array;
}
```

class Unsafe

BitConverter.GetBytes(ulong value)
.NET 9.0 vs .NET Framework 4.7.2

Method	Job	Runtime	Mean	Ratio	Code Size
BitConverterGetBytes	.NET 9.0	.NET 9.0	7.927 ns	0.86	58 B
BitConverterGetBytes	.NET Framework 4.7.2	.NET Framework 4.7.2	9.295 ns	1.00	114 B



class Unsafe

```
ref TTo Unsafe.As<TFrom, TTo>(ref TFrom source)
```

```
.NET 9.0
// class BitConverter
public static long BitConverter.ToInt64(byte[] value, int startIndex)
{
    if (value == null)
        ThrowHelper.ThrowArgumentNullException(...);

    if (unchecked((uint)startIndex) >= unchecked((uint)value.Length))
        ThrowHelper.ThrowArgumentOutOfRangeException(...);

    if (startIndex > value.Length - sizeof(long))
        ThrowHelper.ThrowArgumentException(...);

    return Unsafe.ReadUnaligned<long>(ref value[startIndex]);
}
```

```
public static T ReadUnaligned<T>(scoped ref readonly byte source)
{
    return As<byte, T>(ref Unsafe.AsRef(in source));
}
```

class Unsafe

```
ref TTo Unsafe.As<TFrom, TTo>(ref TFrom source)
```

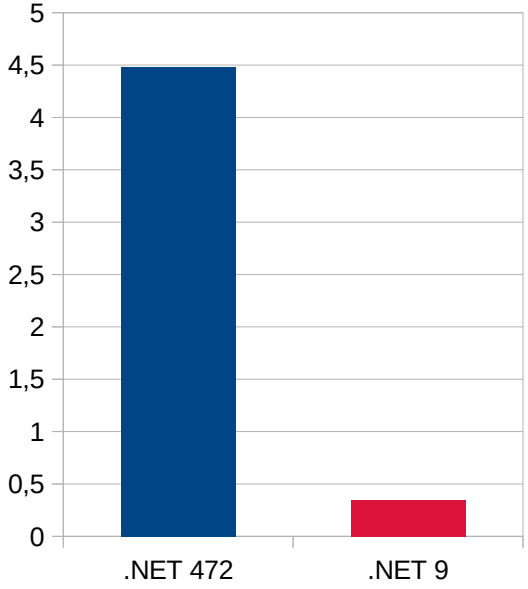
.NET Framework 4.7.2

```
// class BitConverter
public unsafe static long BitConverter.ToInt64(byte[] value, int startIndex)
{
   // ……

    fixed (byte* ptr = &value[startIndex])
    {
        if (startIndex % 8 == 0)
        {
            return *(long*)ptr;
        }

        int num = *ptr | (ptr[1] << 8) | (ptr[2] << 16) | (ptr[3] << 24);
        int num2 = ptr[4] | (ptr[5] << 8) | (ptr[6] << 16) | (ptr[7] << 24);
        return (uint)num | ((long)num2 << 32);
    }
}
```

class Unsafe

BitConverter.ToInt64(byte[] value, int startIndex)
.NET 9.0 vs .NET Framework 4.7.2

Method	Job	Runtime	Mean	Ratio	Code Size
BitConverterToInt64	.NET 9.0	.NET 9.0	0.3479 ns	0.08	84 B
BitConverterToInt64	.NET Framework 4.7.2	.NET Framework 4.7.2	4.4812 ns	1.00	363 B



class Unsafe

```
ref TTo Unsafe.As<TFrom, TTo>(ref TFrom source)
```

```
int i = 1;
string str = string.Format($"a{i}");

ref string refStr = ref str;
ref IntPtr refLong = ref Unsafe.As<string, IntPtr>(ref refStr);

Console.WriteLine($"str address: {refLong:X}") ;
```

```
str address: 1B35B8F6610
```

```
GC.Collect(2, GCCollectionMode.Forced);
Console.WriteLine($"GC.Collect");
Console.WriteLine($"str address: {refLong:X}");
```

```
str address: 1B35B8F6610
GC.Collect
str address: 1B35B05DF88
```

class Unsafe

```
ref T Unbox<T>(object box)
```

```
object obj = 3;
Console.WriteLine($"obj = {obj}"); // obj = 3
ref int refI = ref Unsafe.Unbox<int>(obj);
refI++;
Console.WriteLine($"obj = {obj}"); // obj = 4
```

class Unsafe

```
ref T Unbox<T>(object box)
```

```
Dictionary<string, object> dict = new();

object val = dict["color"]; // ColorArgb

ColorArgb color = (ColorArgb)val;
color.R = 255;
dict["color"] = color;
ref ColorArgb color = ref Unsafe.Unbox<ColorArgb>(val);
color.R = 255;
```

Danger!

```
public static void Dump()
{
    long[] buffer = [1];
    ref long refBuffer0 = ref buffer[0];

    for (int i = 0; i < 1_000_000; i++)
    {
        long data = Unsafe.Add(ref refBuffer0, i); // read next 8 bytes of memory

        // save data
    }
}


try { Dump(); } // AccessViolationException
catch (Exception e)
{
    Console.WriteLine(e.Message); // never catches!
}
```

Danger!

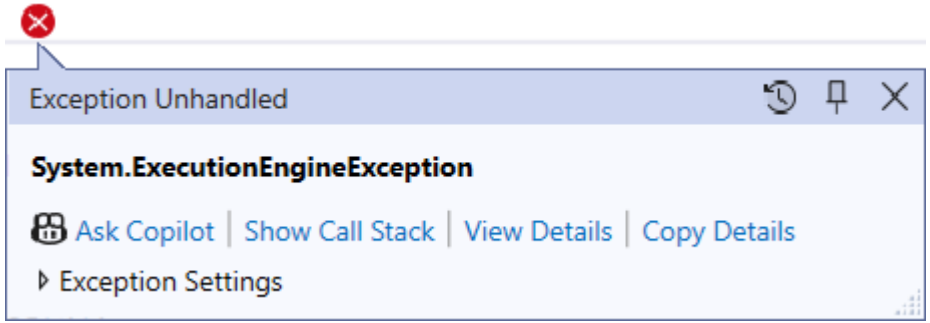
```
public static void BufferOverrun()
{
    byte[] bytes1 = [1];     // <= 1
    byte[] bytes2 = new byte[100];

    ref long refBuf = ref Unsafe.As<byte, long>(ref bytes1[0]);

    Unsafe.Add(ref refBuf, 4) = 0x1122334455667788;  // !!!
    Unsafe.Add(ref refBuf, 3) = 0x0102030405060708;  // !!!

    var x = bytes2[0];
    // x = 0x88
    GC.Collect();
}
```

bytes2

Name	Value
bytes2	{byte[0x00000064]}
[0]	0x88
[1]	0x77
[2]	0x66
[3]	0x55
[4]	0x44
[5]	0x33
[6]	0x22
[7]	0x11
[8]	0x00
[9]	0x00
[10]	0x00

Danger!

```
public static void BufferOverrun()
{
    byte[] bytes1 = [1];
    byte[] bytes2 = new byte[100];

    ref long refBuf = ref Unsafe.As<byte, long>(ref bytes1[0]);

    Unsafe.Add(ref refBuf, 4) = 0x1122334455667788;
    Unsafe.Add(ref refBuf, 3) = 0x0102030405060708;

    var x = bytes2[0];
    // x = 0x88
    GC.Collect();
}
```



Проблематика

+ Использование управляемых указателей позволяет напрямую работать с памятью и оптимизировать код

– Операции класса Unsafe с управляемыми указателями небезопасны

Решение: спрятать управляемый указатель вместе с размером блока в ссылочной структуре, которая реализует безопасные функции

ref struct Span<T> C# 7.3 (2018) - интерфейс над непрерывными областями произвольной управляемой или нативной памяти

```
public readonly ref struct Span<T>
{
    /// <summary>A byref or a native ptr.</summary>
    internal readonly ByReference<T> _pointer;

    /// <summary>The number of elements this Span contains.</summary>
    private readonly int _length;

}
```

ref struct

- Структура с модификатором ref – value type (переменные сохраняются по значению)
- Ограничение: объекты типа ref-структур могут храниться **только в стеке (или в регистрах)**

ref struct

- Может (но не обязана!) содержать поля-ссылки (**ref** fields)
- Может быть локальной переменной (кроме async-функций до C# 13)
- Может быть полем другой **ref**-структуры

```
ref struct Pax
{
    public Span<char> Name;
}
```

- Можно передавать как параметр в функцию
- Можно возвращать по значению из функции

```
static Span<char> Method(Span<char> t)
{
    return t;
}
```

ref struct

- Не может быть полем класса

```
class Pax
{
    public Span<char> Name;
}
```

- Не может быть элементом массива
- Нельзя присваивать значение переменной типа Object или интерфейс

```
object? span = "number 123456".AsSpan()
```

ref struct

- До C# 13 нельзя было :
  - использовать в качестве аргумента дженерик-функций
  - реализовывать интерфейсы
  - использовать в async-функциях

ref field C# 11 (.NET 7.0)

```
public readonly ref struct Span<T>
{
    /// <summary>A byref or a native ptr.</summary>
    internal readonly ref T _reference;

    /// <summary>The number of elements this Span contains.</summary>
    private readonly int _length;

}
```

ref struct Span<T> (.NET 6.0 and earlier)

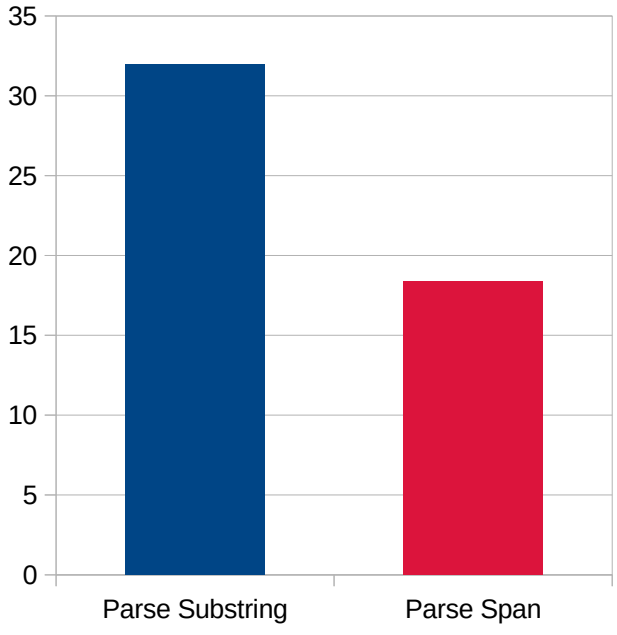
```
public readonly ref struct Span<T>
{
    /// <summary>A byref or a native ptr.</summary>
    internal readonly ByReference<T> _pointer;

    /// <summary>The number of elements this Span contains.</summary>
    private readonly int _length;

}

internal readonly ref struct ByReference<T>
{
#pragma warning disable CA1823, 169 // private field '{blah}' is never used
    private readonly IntPtr _value;
#pragma warning restore CA1823, 169
}
```

ReadOnlySpan<T>

```
string str = "number 123456";
var subStr = str.Substring(7);  // subStr == "123456"
int.TryParse(subStr, out var number); // number == 123456

string str = "number 123456";
ReadOnlySpan<char> span = str.AsSpan(7);  // span == "123456"
int.TryParse(span, out var number); // number == 123456
```

ReadOnlySpan<T>

Method	Mean	Ratio	Gen0	Code Size	Allocated	Alloc Ratio
ParseIntSubstr	31.95 ns	1.00	0.0127	164 B	40 B	1.00
ParseIntSpan	18.39 ns	0.58	-	182 B	-	0.00



ReadOnlySpan<T>

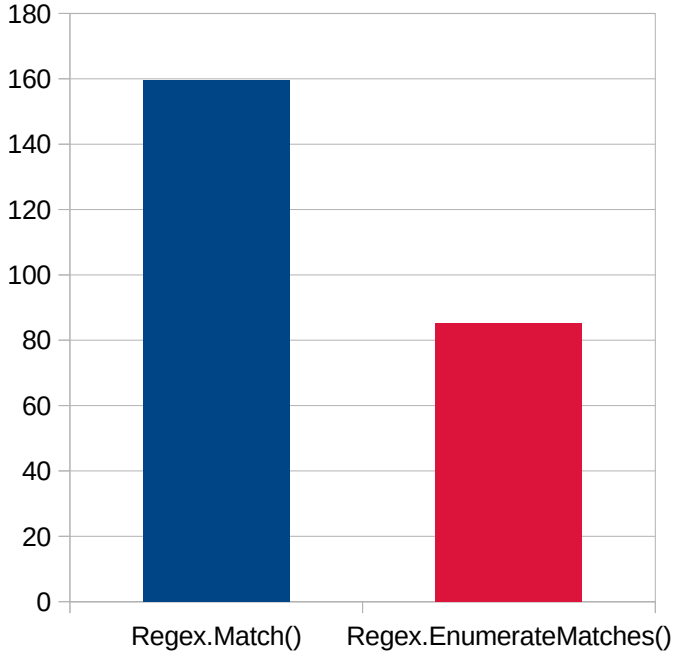
```
private static Regex regex = new Regex(@"\d+", RegexOptions.Compiled);

public static int? ParseIntSubstring(string str)
{
    Match match = regex.Match(str);
    if (match.Success)
    {
        int.TryParse(match.Value, out int result);
    }

    return null;
}
```

ReadOnlySpan<T>

```
private static Regex regex = new Regex(@"\d+", RegexOptions.Compiled);

public static int? ParseIntSpan(string str)
{
    Regex.ValueMatchEnumerator matches = regex.EnumerateMatches(str);

    foreach (ValueMatch match in matches)
    {
        if (int.TryParse(str.AsSpan(match.Index, match.Length), out int result))
        {
            return result;
        }
    }

    return null;
}
```

Regex.Match() vs Regex.EnumerateMatches()

Method	Mean	Ratio	Gen0	Code Size	Allocated	Alloc Ratio
ParseIntSubstr	159.46 ns	1.00	0.0789	191 B	248 B	1.00
ParseIntSpan	85.13 ns	0.53	-	401 B	-	0.00



ref struct ValueMatchEnumerator

```
public ref struct ValueMatchEnumerator  //: IEnumerator
{
    private readonly Regex _regex;
    private readonly ReadOnlySpan<char> _input;
    private ValueMatch _current;
    private int _startAt;
    private int _prevLen;

    public readonly ValueMatchEnumerator GetEnumerator() => this;

    public readonly ValueMatch Current => _current;

    public bool MoveNext() { …. }

}

public interface IEnumerator<T>
{
    bool MoveNext();

    T Current { get; }

    void Reset();
}
```

ref struct ValueMatch

```
public readonly ref struct ValueMatch
{
    private readonly int _index;
    private readonly int _length;

    internal ValueMatch(int index, int length)
    {
        _index = index;
        _length = length;
    }

    public int Index => _index;

    public int Length => _length;
}
```

Под капотом

```
public static ReadOnlySpan<char> AsSpan(this string? text)
{
    if (text == null)
        return default;

    return new ReadOnlySpan<char>(ref text.GetRawStringData(), text.Length);
}
```

```
public ReadOnlySpan<T> Slice(int start, int length)
{
    if (start + length > _length)
        ThrowHelper.ThrowArgumentOutOfRangeException();

    return new ReadOnlySpan<T>(ref Unsafe.Add(ref _reference, start), length);
}
```

Под капотом

```
public static ReadOnlySpan<char> AsSpan(this string? text, int start, int length)
{
    if (text == null)
    {
        if (start != 0 || length != 0)
            ThrowHelper.ThrowArgumentOutOfRangeException(ExceptionArgument.start);
        return default;
    }

    if (start + length > text.Length)
        ThrowHelper.ThrowArgumentOutOfRangeException(ExceptionArgument.start);

    return new ReadOnlySpan<char>(ref Unsafe.Add(ref text.GetRawStringData(), start),
                                  length);
}
```

Под капотом

```
public static bool TryParse(ReadOnlySpan<char> s, NumberStyles style, ..., out int result)
{
    NumberFormatInfo.ValidateParseStyleInteger(style);
    return Number.TryParseBinaryInteger(s, style, ..., out result) == ParsingStatus.OK;
}


public static bool TryParse(string? s, NumberStyles style, ..., out int result)
{
    NumberFormatInfo.ValidateParseStyleInteger(style);

    if (s is null)
    {
        result = 0;
        return false;
    }

    return Number.TryParseBinaryInteger(s.AsSpan(), style, ..., out result) == ParsingStatus.OK;
}
```

Под капотом

```
public static bool TryParse(ReadOnlySpan<byte> utf8Text, NumberStyles style,... out long result)
{
    NumberFormatInfo.ValidateParseStyleInteger(style);
    return Number.TryParseBinaryInteger(utf8Text, style,…, out result) == ParsingStatus.OK;
}
```

Под капотом

```
public int IndexOf(string source, string value, int startIndex, int count, options)
{
    if (source == null)
        ThrowHelper.ThrowArgumentNullException(ExceptionArgument.source);

    if (value == null)
        ThrowHelper.ThrowArgumentNullException(ExceptionArgument.value);

    if (!source.TryGetSpan(startIndex, count, out ReadOnlySpan<char> sourceSpan))
    {
        ThrowHelper.ThrowArgumentOutOfRangeException(...);
    }

    int result = IndexOf(sourceSpan, value, options);
    if (result >= 0)
    {
        result += startIndex;
    }

    return result;
}
```

Под капотом

```
public int Compare(string? string1, string? string2, CompareOptions options)
{
    int retVal;

    if (string1 == null)
    {
        retVal = (string2 == null) ? 0 : -1;
        return retVal;
    }
    if (string2 == null)
    {
        retVal = 1;
        return retVal;
    }

    return Compare(string1.AsSpan(), string2.AsSpan(), options);
}
```

Вывод

Классы Span<T> и ReadOnlySpan<T> – не просто обёртка или дополнение, а один из основных базовых классов-"кирпичиков" DotNet runtime.

Создание Span<T>

```
Span<byte> span1 = new byte[256];
UseSpan(span1);


Span<byte> span2 = stackalloc byte[256];
UseSpan(span2);


var buffer = ArrayPool<byte>.Shared.Rent(256);
Span<byte> span = buffer;
UseSpan(span);
ArrayPool<byte>.Shared.Return(buffer);


nint native = Marshal.AllocHGlobal(N);
Span<byte> nativeSpan = new Span<byte>(native.ToPointer(), N);
UseSpan(nativeSpan);
Marshal.FreeHGlobal(native);
```

Span<T>

```
public Span<byte> GetArraySlice(int i)
{
    byte[] bytes = new byte[] { 0, 1, 2, 3, 4, 5, 6, 7, 8, 9 };
    return bytes.AsSpan(i);
}
```

stackalloc Span<T>

```
public Span<byte> GetRefStackRef(int i)
{
    [ScopedRef]
    Span<byte> bytes = stackalloc byte[256];
    return bytes;
}
```

CS8352 Cannot use variable 'bytes' in this context because it may expose referenced variables outside of their declaration scope

stackalloc vs ArrayPool.Rent()

```
// class INumberBase<TSelf>

static virtual TSelf Parse(ReadOnlySpan<byte> utf8Text, NumberStyles style,
                                                        IFormatProvider? provider)
{
    // Convert text using stackalloc for <= 256 characters and ArrayPool otherwise
    char[]? utf16TextArray;
    scoped Span<char> utf16Text;
    int textMaxCharCount = Encoding.UTF8.GetMaxCharCount(utf8Text.Length);

    if (textMaxCharCount <= 256)
    {
        utf16TextArray = null;
        utf16Text = stackalloc char[256];
    }
    else
    {
        utf16TextArray = ArrayPool<char>.Shared.Rent(textMaxCharCount);
        utf16Text = utf16TextArray;
    }
```

ArrayPoolSpanHelper<T>

```
public ref struct ArrayPoolSpanHelper<T> // : IDisposable
{
    private T[] _data;
    private int _size;

    public Span<T> SpanScoped => _data.AsSpan(0, _size);

    public ArrayPoolSpanHelper(int size)
    {
        _data = ArrayPool<T>.Shared.Rent(size);
        _size = size;
    }

    public void Dispose()
    {
        _size = 0;
        ArrayPool<T>.Shared.Return(_data, true);
    }
}
```

ArrayPoolSpanHelper<T>

```
public void UseArrayPool()
{
    using scoped var helper = new ArrayPoolSpanHelper<byte>(N);
    F(helper.SpanScoped);
}
```

Span<T> Format()

```
int id = 123;
DateTime date = DateTime.Now;
string message = "This is a log message.";

string text = string.Format("{0:D6} {1:yyyy-MM-dd HH:mm:ss} {2}", id, date, message);

string text2 = $"{id:D6} {date:yyyy-MM-dd HH:mm:ss} {message}";
```

Span<T>

```
public ref struct SpanStringBuilder
{
    private Span<char> _buffer;
    private int _position;

    public Span<char> Value => _buffer.Slice(0, _position);

    public SpanStringBuilder(Span<char> buffer)
    {
        _buffer = buffer;
        _position = 0;
    }

    public void Append(string value)
    {
        if (_position + value.Length > _buffer.Length)
            throw new InvalidOperationException("Buffer overflow");
        value.AsSpan().CopyTo(_buffer.Slice(_position)); // value >> _buffer
        _position += value.Length;
    }
```

Span<T>

```
public void Append(char value)
{
    if (_position >= _buffer.Length) { throw … }
    _buffer[_position++] = value;
}

public void Append<T>(T value, string? format = null, IFormatProvider? provider = null)
    where T : ISpanFormattable
{
    if (value.TryFormat(_buffer.Slice(_position), out int written, format))
    {
        _position += written;
    } else { throw … }
}
```

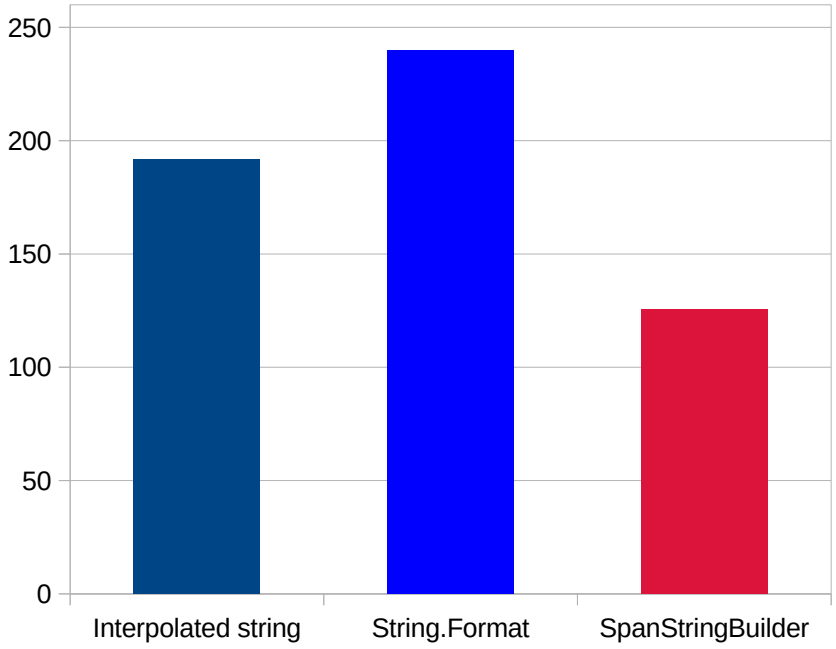
```
public interface ISpanFormattable : IFormattable
{
    bool TryFormat(Span<char> destination,
                   out int charsWritten,
                     ReadOnlySpan<char> format,
                     IFormatProvider? provider);
}
```

Span<T> Format()

```
int id = 123;
DateTime date = DateTime.Now;
string message = "This is a log message.";

using scoped var helper = new ArrayPoolSpanHelper<char>(1024);
scoped Span<char> chars = helper.SpanScoped;
// Span<char> chars = stackalloc char[1024];

scoped SpanStringBuilder spanStringBuilder = new SpanStringBuilder(chars);

// var text = $"{id:D6} {date:yyyy-MM-dd HH:mm:ss} {message}";
spanStringBuilder.Append(id, "D4");
spanStringBuilder.Append(' ');
spanStringBuilder.Append(date, "yyyy-MM-dd HH:mm:ss");
spanStringBuilder.Append(' ');
spanStringBuilder.Append(message);

ReadOnlySpan<char> textFromSpan = spanStringBuilder.Value;
// "000123 2025-08-24 13:57:04 This is a log message."
```

Span<T> Format()

Method	Mean	Ratio	Gen0	Code Size	Allocated	Alloc Ratio
Interpolated-string	191.9 ns	1.00	0.0381	1,803 B	120 B	1.00
string.Format()	235.9 ns	1.23	0.0534	449 B	168 B	1.40
SpanStringBuilder	125.5 ns	0.66	-	5,502 B	-	0.00



UTF-16 => UTF-8

```
public void WriteTextToStream(Stream stream, string text)
{
    byte[] bytes = Encoding.UTF8.GetBytes(text);

    stream.Write(bytes, 0, bytes.Length);
}
```

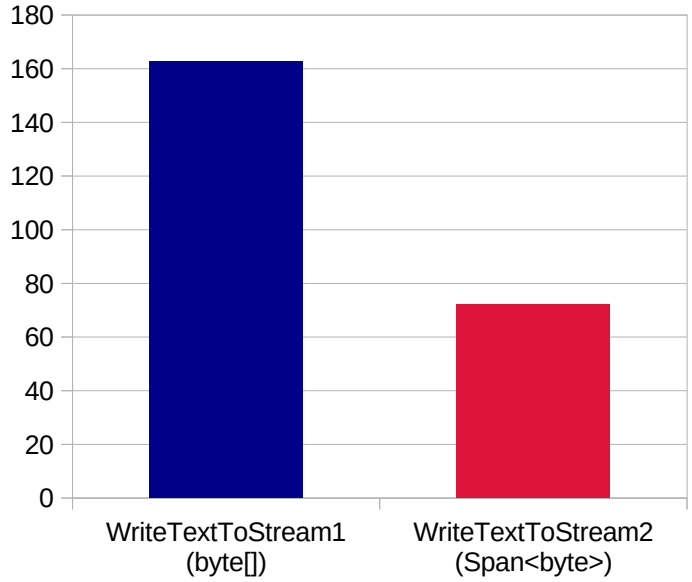
UTF-16 => UTF-8

```
public void WriteTextToStream(Stream stream, Span<char> text)
{
    var maxCount = Encoding.UTF8.GetMaxByteCount(text.Length);

    // rent a temp output buffer in ArrayPoll
    using var helper = new ArrayPoolSpanHelper<byte>(maxCount);
    scoped Span<byte> bytes = helper.SpanScoped;

    var count = Encoding.UTF8.GetBytes(text, bytes);

    stream.Write(bytes.Slice(0, count));
}
```

UTF-16 => UTF-8

WriteTextToStream1 vs `WriteTextToStream2`
`(250 chars to MemoryStream)`

Method	Mean	Ratio	Gen0	Code Size	Allocated	Alloc Ratio
WriteTextToStream-Array	162.52 ns	1.00	0.0892	590 B	280 B	1.00
WriteTextToStream-Span	72.17 ns	0.44	-	4,507 B	-	0.00



MemoryMarshal.Cast<TFrom, TTo>()

```
public void XorArrayBits(byte[] array1, byte[] array2)
{
    int minLength = Math.Min(array1.Length, array2.Length);

    for (var i = 0; i < minLength; i++)
    {
        array1[i] ^= array2[i];
    }
}
```

MemoryMarshal.Cast<TFrom, TTo>()

```
public void XorArrayBits(byte[] array1, byte[] array2)
{
    var spanLongs1 = MemoryMarshal.Cast<byte, ulong>(array1.AsSpan());

    var spanLongs2 = MemoryMarshal.Cast<byte, ulong>(array2.AsSpan());

    int minLengthLongs = Math.Min(spanLongs1.Length, spanLongs2.Length);

    for (var i = 0; i < minLengthLongs; i++)
    {
        spanLongs1[i] ^= spanLongs2[i];
    }

    // rest array1.Length % 8 bytes
    int minLength = Math.Min(array1.Length, array2.Length);
    for (var i = minLengthLongs * 8; i < minLength; i++)
    {
        array1[i] ^= array2[i];
    }
}
```

MemoryMarshal.Cast<TFrom, TTo>()

```
public static unsafe Span<TTo> Cast<TFrom, TTo>(Span<TFrom> span)
    where TFrom : struct where TTo : struct
{
    uint fromSize = (uint)sizeof(TFrom);
    uint toSize = (uint)sizeof(TTo);
    uint fromLength = (uint)span.Length;
    int toLength;
    if (fromSize == toSize)
    {
        toLength = (int)fromLength;
    }
    else
    {
        ulong toLengthUInt64 = (ulong)fromLength * (ulong)fromSize / (ulong)toSize;
        toLength = checked((int)toLengthUInt64);
    }

    return new Span<TTo>(ref Unsafe.As<TFrom, TTo>(ref span._reference), toLength);
}
```

MemoryExtensions.SequenceEqual()

```
public bool BytesEqual(byte[] buf1, byte[] buf2)
{
    return span1.AsSpan().SequenceEqual(buf2.AsSpan());
}
```

MemoryExtensions.SequenceEqual()

```
public static unsafe bool SequenceEqual<T>(this ReadOnlySpan<T> span,
                                           ReadOnlySpan<T> other)
{
    int length = span.Length;
    int otherLength = other.Length;

    if (RuntimeHelpers.IsBitwiseEquatable<T>())
    {
        return length == otherLength &&
            SpanHelpers.SequenceEqual(
                ref Unsafe.As<T, byte>(ref MemoryMarshal.GetReference(span)),
                ref Unsafe.As<T, byte>(ref MemoryMarshal.GetReference(other)),
                ((uint)otherLength) * (nuint)sizeof(T));
    }
```

SpanHelpers.SequenceEqual()

```
public static unsafe bool SequenceEqual(ref byte first, ref byte second, nuint length)
{
    // ……..................
    if (Vector256.IsHardwareAccelerated && length >= (nuint)Vector256<byte>.Count)
    {
        nuint offset = 0;
        nuint lengthToExamine = length - (nuint)Vector256<byte>.Count;

        if (lengthToExamine != 0)
        {
            do
            {
                if (Vector256.LoadUnsafe(ref first, offset) != // first[offset] !=
                    Vector256.LoadUnsafe(ref second, offset))  // second[offset]
                {
                    goto NotEqual;
                }
                offset += (nuint)Vector256<byte>.Count;
            } while (lengthToExamine > offset);
        }
```

Memory<T>

Memory<T>/ReadOnlyMemory<T>

```
public readonly struct Memory<T> : IEquatable<Memory<T>>
{
    private readonly object? _object;
    private readonly int _index;
    private readonly int _length;

    [MethodImpl(MethodImplOptions.AggressiveInlining)]
    public Memory(T[]? array)
    {
        if (array == null)
        {
            this = default;
            return; // returns default
        }

        _object = array;
        _index = 0;
        _length = array.Length;
    }
```

Memory<T>/ReadOnlyMemory<T>

- Обычная структура, хранит ссылку на управляемый массив
- Может быть членом класса
- Можно использовать в async-функциях и итераторах через границу await или yield
- Реализует функцию Slice() и свойство Span

Memory<T>/ReadOnlyMemory<T>

- Не может хранить ссылку на стек
- Упрощённая функциональность, вся работа через Span<T>
- Из Memory<T> можно получить Span<T>
  var span = memory.Span;
- Из Span<T> нельзя получить Memory<T>
  ~~var memory = span;~~

Memory<T>/ReadOnlyMemory<T>

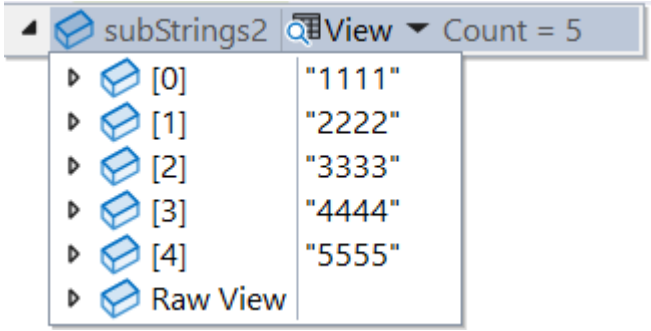
```
public Memory<T> Slice(int start)
{
    if ((uint)start > (uint)_length)
    {
        ThrowHelper.ThrowArgumentOutOfRangeException(ExceptionArgument.start);
    }

    return new Memory<T>(_object, _index + start, _length - start);
}
```

Memory<T> struct

```
public IEnumerable<ReadOnlyMemory<char>> SplitString(string text, char separator)
{
    int startIndex = 0;
    ReadOnlyMemory<char> memStr = text.AsMemory();
    for (int i = 0; i < text.Length; i++)
    {
        if (text[i] == separator || (i == text.Length - 1))
        {
            yield return memStr.Slice(startIndex, i - startIndex);
            startIndex = i + 1;
        }
    }
}
```

Memory<T> struct

```
string str = "1111;2222;3333;4444;5555";

List<ReadOnlyMemory<char>> subStrings = new List<ReadOnlyMemory<char>>();

IEnumerable<ReadOnlyMemory<char>> enumerator = SplitString(str, ';');

foreach (ReadOnlyMemory<char> item in enumerator)
{
    subStrings.Add(item);

    // read int
    int.TryParse(item.Span, out var result);
}
```



Вопрос*

Можно ли изменить память у переменной string?

```
string str = "123456";
```

ChatGPT:
*«В .NET объекты типа string являются **неизменяемыми (immutable)**. Это означает, что после создания объекта string его значение **не может быть изменено**. »*

```
ReadOnlyMemory<char> roMemory = str.AsMemory();

Memory<char> memory = MemoryMarshal.AsMemory(roMemory);

memory.Span[0] = '9';

Console.WriteLine(str); // ?

// "923456";
```

Вопрос*

Можно ли аллоцировать Span<object> на стеке?

```
Span<object?> spanOfString = stackalloc object?[128];

[InlineArray(128)]
public struct InlineArray128<T>
{
    private T element;
}


InlineArray128<object?> array = default;

Span<object?> span = array;

for (int i = 0; i < span.Length; i++)
{
    span[i] = i.ToString();
}
```

Заключение

- **ref-переменные** – это управляемые указатели (managed pointers), они отслеживаются сборщиком мусора.
- **public class Unsafe** позволяет оперировать управляемыми указателями на низком уровне. **Использовать только в runtime!**
- **ref-структура Span<T>** - это интерфейс над непрерывными областями произвольной управляемой или нативной памяти.
- **ref-структуры** позволяют увеличивать производительность кода, уменьшая накладные расходы на выделение и копирование памяти, преобразование данных.
- **структура Memory<T>** дополняет Span<T>, её можно хранить в куче.

Полезные ссылки

Low Level Struct Improvements

https://github.com/dotnet/csharplang/blob/d45a4817470319e08707d2426b9ad4edebe99676/proposals/low-level-struct-improvements.md

C# language specification

16.2.3 **ref modifier**
https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/language-specification/structs#1623-ref-modifier

Спасибо за внимание

Вопросы?

Контакты:

email: yurymalich@yandex.ru

@Yury_Malich

web: http://www.malich.org/duplicate_searcher

XING: https://www.xing.com/profile/Yury_Malich